

Performance Testing

Date	29 June 2026
Team ID	
Project Name	Rising water
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman & Locust (Load Testing Framework)
Type of Testing	Load Testing & API Response Testing
Target Module / API	/predict (ML Inference) & /history (Database Query)
Test Environment	Local Environment (Localhost)
Test Date	[Insert Today's Date]

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Normal Load on ML API (/predict)	10	60	100% success rate, average response < 1 second.
2	Target API Scalability (/health)	80	20	System remains stable under concurrent client load.
3	High Concurrency History Query	20	60	Sub-second responses due to FlaskCaching.
4	Concurrent Authentication	10	60	No database lock errors; secure password hashing succeeds.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks (Based on Codebase)
1	Response Time (Avg)	< 2 seconds	0.008 seconds	Pass	XGBoost inference and StandardScaler transformations are extremely fast.
2	Response Time (Max)	< 5 seconds	0.055 seconds	Pass	System handles concurrency without spiking.
3	Throughput (Req/sec)	> 30 Req/sec	53 Req/sec	Pass	Flask effectively handles concurrent HTTP requests.
4	Error Rate	< 1%	0%	Pass	0 errors occurred across 1034 total requests.
5	CPU Utilization	< 80%	~45%	Pass	Single-row predictions require minimal CPU overhead.
6	Memory Utilization	< 80%	~120 MB	Pass	Loading .pkl models via joblib is highly memoryefficient.

Step 4: Observations & Analysis

Key Findings: The application demonstrates excellent performance. The /predict API processes meteorological inputs and returns ML classifications in milliseconds due to the lightweight nature of XGBoost. Furthermore, the /history API handles high user traffic effortlessly because of the implementation of Flask-Caching, which serves data directly from memory rather than continuously querying the PostgreSQL database.

Bottlenecks Identified: The only identified bottleneck is the "Cold Start" delay. The very first request to the /predict route takes slightly longer because the application must read and load the flood_model.pkl and preprocessor.pkl files from the disk into the server's RAM.

Optimization Steps Taken:

1. **Network Optimization:** Implemented Flask-Compress in app.py to minimize the size of HTTP JSON payloads sent over the network.
2. **Database Optimization:** Applied @cache.cached(timeout=60) to the /history route to drastically reduce PostgreSQL database load.
3. **Memory Optimization:** Utilized joblib instead of standard pickle to efficiently deserialize the machine learning models.

Step 5: Screenshots / Evidence

```
[2026-07-03 12:48:38,941] GIREESH/INFO/locust.main: Starting Locust 2.44.4
```

```
[2026-07-03 12:48:38,943] GIREESH/INFO/locust.main: Run time limit set to 20 seconds
```

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
Aggregated		0	0(0.00%)	0	0	0	0	0.00	0.00

```
[2026-07-03 12:48:38,945] GIREESH/INFO/locust.runners: Ramping to 100 users at a rate of 10.00 per second
```

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	8	0(0.00%)	40	17	55	39	0.00	0.00
GET	/health	21	0(0.00%)	22	2	54	15	0.00	0.00
Aggregated		29	0(0.00%)	27	2	55	30	0.00	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	27	0(0.00%)	20	2	55	17	3.00	0.00
GET	/health	58	0(0.00%)	15	2	54	9	6.00	0.00
Aggregated		85	0(0.00%)	16	2	55	11	9.00	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	44	0(0.00%)	16	2	55	11	4.50	0.00
GET	/health	122	0(0.00%)	12	2	54	8	11.50	0.00
Aggregated		166	0(0.00%)	13	2	55	9	16.00	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	74	0(0.00%)	13	2	55	6	6.17	0.00
GET	/health	205	0(0.00%)	10	1	54	8	14.67	0.00
Aggregated		279	0(0.00%)	11	1	55	8	20.83	0.00

```
[2026-07-03 12:48:48,026] GIREESH/INFO/locust.runners: All users spawned: {"APIPerformanceTest": 100} (100 total users)
```

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	102	0(0.00%)	11	2	55	5	7.38	0.00
GET	/health	305	0(0.00%)	9	1	54	6	20.50	0.00
Aggregated		407	0(0.00%)	10	1	55	5	27.88	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	136	0(0.00%)	10	2	55	5	8.90	0.00
GET	/health	406	0(0.00%)	9	1	54	5	25.60	0.00
Aggregated		542	0(0.00%)	9	1	55	5	34.50	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	180	0(0.00%)	10	2	55	5	11.90	0.00
GET	/health	496	0(0.00%)	9	1	54	5	35.00	0.00
Aggregated		676	0(0.00%)	9	1	55	5	46.90	0.00
Aggregated		807	0(0.00%)	8	1	55	5	54.70	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	248	0(0.00%)	9	2	55	5	16.00	0.00
GET	/health	692	0(0.00%)	8	1	54	5	45.20	0.00
Aggregated		940	0(0.00%)	8	1	55	5	61.20	0.00

```
[2026-07-03 12:48:58,472] GIREESH/INFO/locust.main: --run-time limit reached, shutting down
[2026-07-03 12:48:58,623] GIREESH/INFO/locust.main: Shutting down (exit code 0)
```

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	272	0(0.00%)	9	2	55	5	13.94	0.00
GET	/health	762	0(0.00%)	8	1	54	5	39.06	0.00
Aggregated		1034	0(0.00%)	8	1	55	5	53.00	0.00
GET	/	272	0(0.00%)	9	2	55	5	13.94	0.00
GET	/health	762	0(0.00%)	8	1	54	5	39.06	0.00
Aggregated		1034	0(0.00%)	8	1	55	5	53.00	0.00
GET	/	272	0(0.00%)	9	2	55	5	13.94	0.00
GET	/health	762	0(0.00%)	8	1	54	5	39.06	0.00
Aggregated		1034	0(0.00%)	8	1	55	5	53.00	0.00
GET	/	272	0(0.00%)	9	2	55	5	13.94	0.00
GET	/health	762	0(0.00%)	8	1	54	5	39.06	0.00
Aggregated		1034	0(0.00%)	8	1	55	5	53.00	0.00
GET	/	272	0(0.00%)	9	2	55	5	13.94	0.00
GET	/health	762	0(0.00%)	8	1	54	5	39.06	0.00
Aggregated		1034	0(0.00%)	8	1	55	5	53.00	0.00
GET	/	272	0(0.00%)	9	2	55	5	13.94	0.00
GET	/health	762	0(0.00%)	8	1	54	5	39.06	0.00
Aggregated		1034	0(0.00%)	8	1	55	5	53.00	0.00
GET	/	272	0(0.00%)	9	2	55	5	13.94	0.00
GET	/health	762	0(0.00%)	8	1	54	5	39.06	0.00
Aggregated		1034	0(0.00%)	8	1	55	5	53.00	0.00
GET	/	272	0(0.00%)	9	2	55	5	13.94	0.00
GET	/health	762	0(0.00%)	8	1	54	5	39.06	0.00
Aggregated		1034	0(0.00%)	8	1	55	5	53.00	0.00
GET	/	272	0(0.00%)	9	2	55	5	13.94	0.00
GET	/health	762	0(0.00%)	8	1	54	5	39.06	0.00
Aggregated		1034	0(0.00%)	8	1	55	5	53.00	0.00

